CSA1735 – Artificial Intelligence

Assignment 2 – CO2

Student: Thirulogasundar Y | **Register No:** 192321073

Design, Analyze and Validate an Efficient Heuristic Search Strategy for Smart-City Ambulance Routing

Thirulogasundar Y - 192321073

*Code of Conduct: I, **Thirulogasundar Y (192321073)**, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

Scenario Recap

A smart-city emergency control room dispatches ambulances from the nearest hospital to accident sites, over a road graph of junctions with known GPS coordinates and travel-time costs on every road. As the service expands from one city to the whole district and state, the road graph grows **1,000 → 10,000 → 100,000 → 1,000,000+** junctions, and the control room must still find the fastest route, with every preprocessing step included in the efficiency analysis.

TASK 1 – Design the Agent and the Heuristic Search Algorithms (5 Marks)

PEAS description

Element	Specification
Performance measure	Minimum total travel time from hospital to accident site; patient survival/outcome; keeping up with hundreds of requests per minute city-wide
Environment	The city/district/state road graph: junctions, roads, traffic signals, other emergency and civilian vehicles
Actuators	The ambulance's steering/throttle (or, for the routing server, the dispatch instruction sent to the driver/autonomous vehicle)
Sensors	GPS for the ambulance's live position, the road graph/map database, live traffic or closure feeds where available

Environment classification, each justified

Property	Classification	Justification
Observable	**Partially observable** (the routing server's map is	The control room knows the static graph but not live obstructions, accidents on the route, or real-time congestion unless explicitly fed in

Thirulogasundar Y - 2232107

Property	Classification	Justification
	a model, not a live view of every road's current condition)	
Deterministic	**Stochastic**	Traffic, signals and other drivers mean the same route can take different actual times on different attempts
Episodic / Sequential	**Sequential**	Each road segment driven affects the remaining route and the time left, so decisions are chained, not independent
Static / Dynamic	**Dynamic**	Traffic conditions, other emergency vehicles and (per the Day-4 challenge) road closures can change while the ambulance is already travelling
Discrete / Continuous	**Discrete**	The road graph, junctions and the route-planning decisions are modelled as a finite graph with discrete edges, even though real GPS coordinates are continuous-valued inputs to the heuristic
Single / Multi-agent	**Multi-agent**	Other traffic (and other ambulances dispatched by the same control room) share the same roads and affect actual travel time

Thirulogasundary - 192321073

Heuristic functions, with admissibility and consistency proofs

$h_1(n)$ = straight_line_distance(n, goal) / MAX_SPEED (the fastest speed permitted anywhere on the network).

- **Derivation from a relaxed problem:** relax the real road network so the ambulance can travel in a straight line, off-road, at the network's fastest

permitted speed, ignoring the actual road graph entirely — the cost of this relaxed problem's optimal solution is exactly h_1.

- **Admissibility proof:** every real edge cost is built as cost(n, n') = distance(n, n') / speed, where speed ≤ MAX_SPEED for every road, so cost(n, n') ≥ distance(n, n') / MAX_SPEED for every edge. Summed along any real path from n to the goal, the total real cost is therefore always ≥ the straight-line distance from n to the goal divided by MAX_SPEED — which is exactly $h_1(n)$. So $h_1(n)$ never overestimates the true remaining cost: **h_1 is admissible.**
- **Consistency proof:** for any edge (n, n'), $h_1(n)$ = dist(n, goal)/MAX_SPEED ≤ [dist(n, n') + dist(n', goal)]/MAX_SPEED (triangle inequality on Euclidean distance) = dist(n,n')/MAX_SPEED + $h_1(n')$ ≤ cost(n,n') + $h_1(n')$ (since cost(n,n') ≥ dist(n,n')/MAX_SPEED as shown above). This is exactly the consistency condition h(n) ≤ cost(n,n') + h(n'), so **h_1 is consistent.**

$h_2(n) = 0.5 \times h_1(n)$ (a deliberately weakened, less-informed heuristic).

- **Derivation:** relax the problem the same way as for h_1, but additionally (and pessimistically) assume the ambulance is only allowed to use half of MAX_SPEED's benefit — i.e. a looser relaxation that still never overestimates since it only shrinks an already-admissible estimate.
- **Admissibility:** since h_1 is admissible ($h_1 \le$ true cost) and $h_2 = 0.5 \cdot h_1 \le h_1 \le$ true cost, **h_2 is admissible.**
- **Consistency:** for any edge, $h_2(n) = 0.5 \cdot h_1(n) \le 0.5 \cdot [\text{cost}(n,n') + h_1(n')] \le \text{cost}(n,n') + 0.5 \cdot h_1(n') = \text{cost}(n,n') + h_2(n')$ (using h_1's consistency and that $0.5 \cdot \text{cost}(n,n') \le \text{cost}(n,n')$ for non-negative costs), so **h_2 is consistent.**
- **h_1 dominates h_2** ($h_1(n) \ge h_2(n)$ everywhere by construction), so h_1 is the more informed heuristic and is expected, by the dominance property, to cause A* to expand no more nodes than h_2 does on the same problem — verified experimentally in Task 9–10.

Problem formulation

Element	Definition
States	The ambulance's current road junction
Initial state	The dispatching hospital's junction

Thirulokkundar Y - 192211013

Element	Definition
Actions	Drive(junction → adjacent_junction) for every road leaving the current junction
Transition model	Result(s, Drive(s→s')) = s' — driving a road deterministically moves the ambulance to the connected junction (idealised for the base search model; the Day-4 challenge relaxes this)
Goal test	s = accident_site
Path cost	g(n) = Σ travel_time(road) along the roads driven so far

Three meaningfully different informed search strategies

I selected **Greedy Best-First Search (iterative, priority queue on h(n) alone)**, **A* (iterative, priority queue on f(n) = g(n) + h(n))** and **IDA* (outer loop iterative over f-limits; the contour search at each limit is recursive)**. These differ in more than iterative-vs-recursive implementation of the same idea: Greedy ignores path cost so far entirely, A* balances cost-so-far against the heuristic in a single global priority queue, and IDA* re-derives the same f-ordering through repeated bounded recursive passes instead of a persistent queue — three distinct evaluation/search strategies.

1. Greedy Best-First Search — f(n) = h(n) Basic idea: always expand the node that *looks* closest to the goal, ignoring how far it has already travelled; fast and often reasonable, but not optimal, since a node with a small h(n) might sit at the end of a very expensive path.

```
function GREEDY(graph, start, goal):
    open = PriorityQueue(key = h(n)); open.push(start)
    came_from = {start: None}; closed = {}
    while open is not empty:
        node = open.pop_min()
        if node in closed: continue
        closed.add(node)
        if node == goal: return reconstruct_path(came_from, goal)
        for each neighbour of node:
```

Nilogasundar Y, 192321073

```
        if neighbour not in closed and neighbour not in came_from:
            came_from[neighbour] = node
            open.push(neighbour, priority = h(neighbour))
    return failure
```

Justification: establishes a fast, low-overhead baseline to contrast against A*'s optimality guarantee — useful if the control room ever needs a quick, "good enough" route under extreme time pressure.

2. A* — f(n) = g(n) + h(n) Basic idea: expand the node with the lowest *total* estimated cost (cost already incurred plus estimated cost remaining); with an admissible heuristic, this guarantees the first time the goal is popped, the route found is optimal.

```
function A_STAR(graph, start, goal):
    open = PriorityQueue(key = g(n) + h(n)); open.push(start, g=0)
    best_g = {start: 0}; came_from = {start: None}; closed = {}
    while open is not empty:
        node, g = open.pop_min()
        if node in closed: continue
        closed.add(node)
        if node == goal: return reconstruct_path(came_from, goal)
        for each (neighbour, cost) of node:
            new_g = g + cost
            if new_g < best_g.get(neighbour, infinity):
                best_g[neighbour] = new_g; came_from[neighbour] = node
                open.push(neighbour, priority = new_g + h(neighbour), g = new_g)
    return failure
```

Justification: the central strategy under investigation — it is the natural choice whenever a *guaranteed-fastest* route matters, which is exactly the ambulance-routing performance measure.

3. IDA* — outer loop iterative; the bounded contour search is recursive Basic idea: run a depth-first search that only explores nodes with f(n) ≤ a limit, starting the limit at h(start); whenever a pass fails, raise the limit to the smallest f-value that exceeded it and try again — combining A*'s optimality with DFS's linear memory.

```
function IDA_STAR(graph, start, goal):
    limit = h(start)
    loop:
        result = CONTOUR(start, g=0, limit)              # RECURSIVE
        if result == FOUND: return reconstruct_path()
        if result == infinity: return failure
        limit = result

function CONTOUR(node, g, limit):                        # RECURSIVE
    f = g + h(node)
    if f > limit: return f
    if node == goal: return FOUND
    min_exceed = infinity
    for each (neighbour, cost) of node not on current path:
        result = CONTOUR(neighbour, g + cost, limit)
        if result == FOUND: return FOUND
        min_exceed = min(min_exceed, result)
    return min_exceed
```

Justification: the natural answer to "can we get A*'s optimality without A*'s memory?" — essential to evaluate given the stated requirement that the solution must remain practical as the road graph passes 1,000,000+ junctions, where A*'s open/closed lists could become a genuine memory concern.

Preprocessing required

- **Adjacency-list construction:** O(V + E), built once for the whole graph before any search runs.
- **Heuristic pre-computation:** none of the per-node straight-line distances need to be pre-computed and cached for a single query (they are O(1) to compute on demand from coordinates), but the **junction coordinate table itself** (O(n) space) must be loaded/available before any heuristic evaluation can occur.
- **Priority-queue structure:** both Greedy and A* require a binary-heap-backed priority queue supporting O(log n) insertion and extraction; IDA* needs no persistent priority queue at all, only the current recursion stack — a

Thirulogasundar

direct preprocessing/data-structure trade-off that the Task 3 complexity derivation makes precise.

TASK 2 – Identify the Basic Operation (2 Marks)

For all three algorithms the basic operation is **node expansion paired with its heuristic evaluation**: popping a node from the frontier (queue or recursion), computing h(n) for it/its neighbours, and generating its successors. Priority-queue insertion/extraction is the mechanism that *orders* this operation, not a separate cost driver, since every insertion is paired 1:1 with a node generation.

Algorithm	Where it occurs	Why appropriate	How the count changes with graph size and heuristic quality
Greedy	open.pop_min() by h(n), then one heuristic evaluation per newly generated neighbour	Greedy's entire behaviour — which branch it commits to — is driven purely by how many nodes it must examine before stumbling onto the goal via the locally-best-looking choice at each step	Can be nearly independent of graph size if the heuristic reliably points toward the goal (few expansions); grows sharply if the heuristic is misleading in some region, since Greedy has no cost-so-far to correct a bad early choice
A*	open.pop_min() by f(n)=g(n)+h(n), then one heuristic evaluation per newly	A*'s cost is exactly the number of nodes with f(n) < C* (the optimal cost) that must be expanded before the goal is popped — node expansion count is the	Grows with the number of nodes whose f-value falls below the optimal cost; a more accurate heuristic shrinks this set directly (fewer nodes look "promising enough" to expand), while a less informed heuristic (closer to

Thirulogasundar Y - 3321073

Algorithm	Where it occurs	Why appropriate	How the count changes with graph size and heuristic quality
	generated/relaxed neighbour	textbook A* complexity measure	h=0) grows it toward uniform-cost search's node count
IDA*	CONTOUR()'s per-call heuristic evaluation and recursive-call generation, repeated once per f-limit	IDA*'s total cost is dominated by repeatedly regenerating the same set of "f(n) ≤ limit" nodes at every contour, so counting node generations captures both the useful work and the redundant-contour overhead	Grows with the number of **distinct f-values** below C* (since each distinct value can force a new, separate contour pass) — with integer step costs this is bounded nicely, but with continuous, real-valued travel times (as in this road network) it can blow up sharply, directly measured in Task 8–9

TASK 3 – Derive the Time- and Space-Complexity Functions (6 Marks)

Notation: b = branching factor, d = solution depth, b* = effective branching factor (observed, see Task 9), C* = optimal solution cost, ε = minimum edge (step) cost, (h* − h) = heuristic error (how far below the true remaining cost h falls). n = total junctions (preprocessing only).

Preprocessing cost (common to all three)

Adjacency-list construction: **O(V + E) = O(n)** for bounded average branching. The coordinate table needed for every heuristic evaluation is also O(n) to hold, built once.

Thirulogasundari Y - 192321073

Greedy Best-First Search

Greedy expands, in the worst case, every node whose h-value happens to look promising enough to be tried before the goal is reached — with a poor heuristic this can be the whole reachable graph:

$$\text{Nodes expanded (worst case)} = O(b^{m})$$

where m is the graph's maximum depth, exactly as for uninformed DFS, since nothing in Greedy's rule prevents it from being led arbitrarily deep by a locally-attractive but globally wrong h-value. With a very informative heuristic, the best case collapses toward **O(d)** (expanding only the nodes on the eventual path). **Space:** the open list can hold up to O(b^m) nodes in the worst case (every generated-but-unexpanded alternative), so **Space = O(b^m)** worst case, though in practice (and as measured in Task 9) it stays much smaller on an informative heuristic. **Combined with preprocessing: O(n + b^m)**.

A*

A* expands exactly the set of nodes n with $f(n) < C^*$ (plus possibly some with $f(n) = C^*$), so the node count is governed by how many nodes fall under that cost ceiling:

$$\text{Nodes expanded} = O\big(b^{\,(h^*-h)/\varepsilon}\big)$$

which is the standard A* result: if the heuristic error (h* − h) grows no faster than logarithmically in the true path cost, this stays polynomial; with h = 0 everywhere (the worst case, Task 4), it reduces to uniform-cost search's $O(b^{C/\varepsilon})$. ***Time*** *is this node count times $O(\log n)$ per priority-queue operation (insertion/extraction): $O(b^{(h-h)/\varepsilon} \cdot \log n)$*. **Space:** both the open list (priority queue) and the closed list must be held simultaneously, each bounded by the same node count: *Space = $O(b^{(h-h)/\varepsilon})$, the well-known reason A*'s memory, not its time, is usually the first practical limit reached. Combined with preprocessing: $O(n + b^{(h-h)/\varepsilon} \cdot \log n)$* for time, *$O(n + b^{(h-h)/\varepsilon})$** for space.

IDA*

A single contour pass at limit l generates every node with $f(n) \le l$, i.e. $O(b^{(l-h(start))/\varepsilon})$ nodes (same shape of expression as A*'s bound, truncated at

Thilogasuriyar - 19232073

that contour's limit). IDA* repeats this once per **distinct f-value** encountered on the way up to C*:

$$\text{Nodes generated} = \sum_{\text{distinct } f\text{-values} \le C^*} O\big(b^{(f - h(\text{start}))/\varepsilon}\big)$$

With integer step costs and a reasonably coarse heuristic, the number of distinct f-values is small and this sum, like the IDS case, is dominated by its final (largest) term — giving the same order as A*'s node count. **With continuous, real-valued edge costs** (as in this ambulance network, where cost = distance/speed for a randomly sampled real-valued speed), *every* contour can discover only a handful of new nodes before hitting the next limit, so the number of distinct f-values can approach the total node count itself, giving a **quadratic** $O(N^2)$-style blow-up in the worst case — derived and measured directly in Task 7–9. **Time** is this total generation count (no log-n factor, since there is no priority queue, only direct recursive comparison against the current limit): **O(\Sigma)** as above. **Space** is only the current recursion path: **O(b \cdot d)** — IDA*'s defining advantage over A*. **Combined with preprocessing: O(n + \Sigma)** for time, **O(n + b \cdot d)** for space.

TASK 4 – Best, Average and Worst-Case Behaviour (4 Marks)

Greedy Best-First Search

- **Best case:** the heuristic is perfect, h = h_true, directly pointing to the goal at every junction with no misleading side-branches — nodes expanded is O(d), only the nodes on the final path.
- **Average case:** a heuristic with a small, roughly consistent relative error (such as the spatially-local h_1 measured in Task 9) — Greedy usually heads roughly toward the goal but can take some locally-attractive detours.
- **Worst case:** a misleading heuristic — one where a node that looks close to the goal actually sits behind a long, expensive detour (for example a dead-end cul-de-sac whose straight-line distance to the goal happens to be small) — Greedy commits to it and may explore most of the graph before backing out, giving O(b^m).

Thirulogasundar Y - 192321073

- **Why:** since Greedy never consults g(n), nothing about its rule can detect or correct a locally-good-but-globally-bad choice; the number of nodes expanded is entirely a function of how often the heuristic's local ranking disagrees with the true global ranking.

A*

- **Best case:** h = h_true everywhere (a perfect heuristic) — A* expands only the O(d) nodes on the optimal path, since every other node's f-value is strictly greater than C* and is never explored.
- **Average case:** a heuristic with small, bounded absolute error (h* − h) — nodes expanded stays polynomial in the error, as derived in Task 3.
- **Worst case:** h = 0 everywhere — A* degenerates exactly to Uniform-Cost Search, expanding every node with g(n) < C*, giving O(b^{C*/ε}).
- **Why:** A*'s node count is precisely the number of nodes whose f-value falls below the optimal cost; a perfect heuristic shrinks that set to just the path itself, while h=0 removes all heuristic guidance and the set reverts to everything reachable within the optimal cost radius.

IDA*

- **Best case:** h = h_true — like A*, only O(d) nodes are ever generated, and (since there is then exactly one relevant f-value, C* itself) essentially one contour suffices.
- **Average case:** integer-like or coarsely-clustered step costs with a reasonable heuristic — a moderate number of distinct f-values, each contour discovering a meaningfully larger set of nodes than the last, keeping total work in the same order as A*'s node count (Task 3/7).
- **Worst case: many distinct f-values**, which is exactly what continuous, real-valued travel-time costs produce (as confirmed experimentally in Task 8–9) — each contour may expand only one or a handful of new nodes before hitting the next limit, so the number of contours approaches the total node count, and total work approaches $O(N^2)$.
- **Why:** IDA*'s efficiency assumes each new contour's limit admits a *meaningfully larger* set of nodes than the previous one; when edge costs are continuous and effectively all-distinct, the gap between consecutive

Thirilogasundar Y - 192321073

achievable f-values can be arbitrarily small, so this assumption breaks down and the repeated-contour overhead dominates.

TASK 5 – Asymptotic Analysis (3 Marks)

Algorithm	Time	Space
Greedy	O(b^m) worst case; O(d) best case (no tight Θ, highly heuristic-dependent)	O(b^m) worst case
A*	Θ(b^{(h*−h)/ε} · log n) — tight given a fixed heuristic-error profile	Θ(b^{(h*−h)/ε})
IDA*	O(N²) worst case (continuous costs, many distinct f-values); Θ(b^{(h*−h)/ε}) average/favourable case	Θ(b·d) — the one bound that stays small regardless of heuristic quality

How each behaves as the road network becomes very large

None of these bounds depend directly on n (total junctions) except through preprocessing — they are governed by the **heuristic's error and the branching factor**, so a district or state-wide expansion that only adds junctions *far* from any currently-dispatched route barely changes A*'s or IDA*'s cost for that route. What matters is how (h* − h) and the number of distinct f-values behave as the *relevant portion* of the network (between hospital and accident site) grows.

Which becomes the limiting factor first — memory or time — as the network grows from 1,000 to 1,000,000+ junctions

For A, memory is the first limiting factor, exactly as with BFS in the uninformed case: Θ(b^{(h*-h)/ε}) open+closed lists grow exponentially in the heuristic's error, and the Task 9 experiment shows this concretely — A*'s open-list peak reaches over 100,000 entries at n=1,000,000 even with a well-informed heuristic. *For IDA, time (via repeated contour re-expansion) is the first limiting factor*, not memory — its recursion stack stays at O(b·d) regardless of n (confirmed by Task 9's measured `max_open` staying at 1 throughout, since no persistent frontier is kept),

Thirilogasundar Y - 192210

but its *node-generation count* can explode well before any memory pressure appears, exactly the quadratic blow-up measured directly in Task 8–9 on this continuous-cost road network. **For Greedy, neither resource grows predictably** — its cost is governed entirely by how often the heuristic misleads it in the specific graph region being searched, which can be excellent or terrible independent of n. This means the practical "limiting factor" for a real smart-city deployment is different per algorithm: A* needs a memory budget that scales with the hardest (least-informed-heuristic) query it will ever serve; IDA* needs the routing server to tolerate unpredictable per-query latency; Greedy needs an accepted risk of occasionally very suboptimal (and not just slow) routes.

TASK 6 – Formulate the Recurrence Relation (3 Marks)

The recursive strategy is IDA*'s `CONTOUR` function (reproduced from Task 1):

```
function CONTOUR(node, g, limit):
    f = g + h(node)
    if f > limit: return f
    if node == goal: return FOUND
    min_exceed = infinity
    for each (neighbour, cost) of node not on current path:
        result = CONTOUR(neighbour, g + cost, limit)
        if result == FOUND: return FOUND
        min_exceed = min(min_exceed, result)
    return min_exceed
```

Component	Identification from the code
Size of the original problem	The "budget" remaining under the current contour — informally, how many more f-respecting hops remain before f(node) would exceed `limit`; call the number of such remaining hops l

Component	Identification from the code
Size of the recursive subproblem	Each call to a neighbour reduces the remaining budget by one hop, i.e. the subproblem has size $l - 1$
Number of recursive calls	Up to b calls (one per neighbour not already on the current path), in the worst case where no `FOUND` return short-circuits the loop
Work performed outside the recursive calls	O(1) to compute `f = g + h(node)` and compare against `limit`, plus O(1) per neighbour to check `min_exceed` — constant work per call
Base case	`f > limit` (the budget is exhausted — return the overshoot value in O(1)), and `node == goal` (success — return `FOUND` in O(1))

The recurrence that follows directly from these components:

```
T(l) = b \cdot T(l-1) + O(1), \qquad T(0) = O(1)
```

This is structurally the same shape as the uninformed-search recurrence (CO1, Task 6), since IDA*'s recursive machinery is still a depth-bounded DFS — the difference lies entirely in *how the bound itself is chosen and advanced* (an f-cost limit raised to the next-smallest overshoot value, rather than a fixed integer depth incremented by exactly 1 each outer iteration), which is precisely what Task 7 must now account for.

TASK 7 – Solve the Recurrence Relation (4 Marks)

Method chosen: iteration (unrolling)

Applicable for the same reason as in the uninformed case: a single-term linear recurrence with a constant multiplier b and a unit decrement in problem size,

Thirulogasundar - 19232073

which unrolls cleanly without needing the Master Theorem's divide-by-a-fraction form.

Unrolling to the base case (l → 0, where T(0) = c_0):

$$T(l) = b^{l} c_0 + c \sum_{i=0}^{l-1} b^{i} = O(b^{l})$$

(identical derivation steps to the uninformed-search case: substitute T(l−1), observe the geometric pattern after k unrollings, and unroll fully to the base case — see the CO1 report for the full step-by-step algebra, which applies unchanged here since the recurrence's shape is the same.)

What the result indicates

A single contour pass costs **exponentially** in the number of hops it is allowed to take under the current f-limit — as with the uninformed case, doubling the allowed depth does not double the cost, it squares it.

Extension to IDA*: how the number of distinct f-values changes the total cost

IDA* sums this per-contour cost once **per distinct f-value** k encountered up to C* (not simply once per integer depth, as in uninformed IDS — this is the key difference):

$$T_{\text{IDA*}} = \sum_{k=1}^{K} O(b^{\,l_k})$$

where K is the number of distinct f-values below C* and l_k is the (small, often near-constant) number of *new* hops that contour k reaches beyond the previous one. **When K is small** (coarse, well-separated f-values, as with integer step costs) this sum behaves just like the geometric series in the uninformed case — dominated by its last term, giving the same $O(b^{(h^*-h)/\varepsilon})$ order as A*. **When K approaches the total node count N** — exactly what continuous, real-valued travel-time costs produce, since almost every node then has its own distinct f-value — each term in the sum contributes only a small, roughly linearly-growing amount of *new* work (because each contour typically discovers only one or a few more nodes than the last), and summing a linearly-growing sequence over K ≈ N terms gives:

$$T_{\text{IDA*}} \approx \sum_{k=1}^{N} O(k) = O(N^{2})$$

This is exactly the **quadratic worst case** flagged in Tasks 3–4, and it is precisely what the Task 8–9 experiment reproduces: at n = 10,000 with real-valued costs,

Thirulogasundar T - 19232073

IDA* was observed generating nodes_expanded_this_iteration ≈ iteration_number at every contour — the textbook signature of this $O(N^2)$ pathology, confirmed from real executed code rather than asserted theoretically.

TASK 8 – Implement and Test the Algorithms (4 Marks)

All three algorithms, the road-network generator and both heuristics were implemented in Python 3 exactly as analysed in Tasks 1–7, and actually executed to produce every number in Tasks 9–11.

2321073

Source code (route_search.py)

```
import random, sys, math, heapq
sys.setrecursionlimit(20000)
MAX_SPEED, MIN_SPEED = 80.0, 20.0

def generate_road_network(n, extra_edge_fraction=0.15, seed=0,
candidate_sample=12):
    rnd = random.Random(seed)
    side = math.sqrt(n)
    coords = {i: (rnd.uniform(0, side), rnd.uniform(0, side)) for i in range(n)}
    adj = {i: [] for i in range(n)}
    def dist(a, b):
        ax, ay = coords[a]; bx, by = coords[b]; return math.hypot(ax-bx, ay-by)
    def add_edge(u, v):
        if u == v: return
        speed = rnd.uniform(MIN_SPEED, MAX_SPEED)
        cost = dist(u, v) / speed
        adj[u].append((v, cost)); adj[v].append((u, cost))
    # Spatially-local attachment: nearest of a random candidate sample (not
uniformly random)
    # -- this is what makes straight-line distance an INFORMATIVE heuristic, as in
a real road map
    for i in range(1, n):
        candidates = rnd.sample(range(i), min(candidate_sample, i))
```

```
        nearest = min(candidates, key=lambda j: dist(i, j))
        add_edge(nearest, i)
    for _ in range(int(n * extra_edge_fraction)):
        u = rnd.randint(0, n - 1)
        candidates = rnd.sample(range(n), min(candidate_sample, n - 1))
        v = min((c for c in candidates if c != u), key=lambda j: dist(u, j),
default=None)
        if v is not None: add_edge(u, v)
    return coords, adj

def straight_line_heuristic(coords, node, goal, max_speed=MAX_SPEED,
scale=1.0):
    ax, ay = coords[node]; bx, by = coords[goal]
    return scale * math.hypot(ax - bx, ay - by) / max_speed

def astar(adj, coords, start, goal, h_scale=1.0):          # iterative, f(n)=g(n)+h(n)
    counter = 0; g0 = 0.0; h0 = straight_line_heuristic(coords, start, goal,
scale=h_scale)
    open_heap = [(g0 + h0, counter, start, g0)]
    best_g = {start: g0}; came_from = {start: None}; closed = set()
    nodes_expanded = heuristic_evals = 0; max_open = 1
    while open_heap:
        max_open = max(max_open, len(open_heap))
        f, _, node, g = heapq.heappop(open_heap)
        if node in closed or g > best_g.get(node, float('inf')): continue
        closed.add(node); nodes_expanded += 1
        if node == goal:
            return {'path': reconstruct(came_from, start, goal), 'nodes_expanded':
nodes_expanded,
                    'heuristic_evals': heuristic_evals, 'max_open': max_open, 'found':
True, 'cost': g}
        for (nbr, c) in adj[node]:
            new_g = g + c
            if new_g < best_g.get(nbr, float('inf')):
                best_g[nbr] = new_g; came_from[nbr] = node
```

```
            h = straight_line_heuristic(coords, nbr, goal, scale=h_scale);
heuristic_evals += 1
            counter += 1; heapq.heappush(open_heap, (new_g + h, counter, nbr,
new_g))
   return {'path': None, 'nodes_expanded': nodes_expanded, 'heuristic_evals':
heuristic_evals,
          'max_open': max_open, 'found': False, 'cost': None}

# greedy_best_first(): identical structure to astar(), but the heap key is h(n) alone, g
is not tracked
# ida_star()/ida_star_search(): outer loop iterative over f-limits; CONTOUR search
is RECURSIVE (Task 1/6)
```

Test cases (actually run, exact console output)

Test 1 – reachable goal, start ≠ goal (30-junction network, seed=1, start=0, goal=29):

```
Greedy: {'path': [0, 1, 5, 11, 29], 'nodes_expanded': 6, 'heuristic_evals': 13,
'max_open': 8, 'found': True}
A*:     {'path': [0, 1, 5, 11, 29], 'nodes_expanded': 12, 'heuristic_evals': 17,
'max_open': 7, 'found': True, 'cost': 0.1649}
IDA*:   {'path': [0, 1, 5, 11, 29], 'nodes_expanded': 71, 'heuristic_evals': 122,
'max_open': 5, 'found': True, 'cost': 0.1649}
```

Expected: on a graph where the heuristic is spatially informative, Greedy, A* and IDA* can agree on the same route (unlike the uninformed-search case, Greedy is not *guaranteed* suboptimal — only *not guaranteed optimal*); A* and IDA* must always report the identical optimal cost. **Matches exactly.**

Test 2 – start equal to goal:

```
{'path': [5], 'nodes_expanded': 1, 'heuristic_evals': 1, 'max_open': 1, 'found': True,
'cost': 0.0}
```

Matches the expected immediate, zero-cost, zero-expansion success.

Test 3 – unreachable goal (node 29 isolated):

```
A*:     {'path': None, 'nodes_expanded': 29, 'heuristic_evals': 30, 'max_open': 7,
'found': False, 'cost': None}
```

```
Greedy: {'path': None, 'nodes_expanded': 29, 'heuristic_evals': 29, 'max_open': 9, 'found': False}
IDA*:   {'path': None, 'nodes_expanded': 1275, 'heuristic_evals': 1768, 'max_open': 1, 'found': False, 'cost': None}
```

Matches: all three correctly report failure after exhausting the reachable component; IDA*'s far higher count again confirms the repeated-contour overhead from Task 3/7.

*Test 4 – an inadmissible heuristic breaks A's optimality guarantee** (h_scale = 3.0, overestimating): on a 2,000-junction network, trial search found a genuine counterexample:

```
s=1711 g=346  optimal_cost=1.2607 (admissible h)  inadmissible_result=1.3213 (scale=3 h)
 -> 4.8% WORSE than optimal, despite expanding fewer nodes (296 vs 891)
```

Expected and matches theory exactly: an inadmissible heuristic can cause A* to accept a suboptimal path the moment it overestimates the true remaining cost enough to make a worse-but-already-expanded goal path look better than a still-unexpanded better one — reproduced from real code, not asserted.

Admissibility/consistency verified empirically (not just by the Task 1 proof)

```
Admissibility check (50 random (node, goal) pairs vs true UCS-computed optimal cost): 0 violations
Consistency check (688 edges, h(n) <= cost(n,n') + h(n') for every edge): 0 violations
```

This empirical check corroborates the Task 1 algebraic proof using the actual generated graph and heuristic, not just the abstract construction argument.

All graphs for the main experiment (Task 9) use the **same seed (42)** and the **same start/goal pair** (junction 0 → junction n−1) for every algorithm at every size, so comparisons across algorithms at a given size are on identical road networks.

TASK 9 – Record Experimental Performance (3 Marks)

Actual results (real console output, seed = 42, times averaged over 5 runs except IDA*'s n ≥ 10,000 rows)

n	Algorithm	Nodes expanded	Heuristic evals	Max open/frontier	Found?	Cost	Optimal?	Avg time (s)
1,000	Greedy	143	170	90	Yes	1.142	No (optimal = 1.030)	0.000247
1,000	A*	371	464	234	Yes	1.030	Yes	0.000742
1,000	IDA*	2,346	3,672	15	Yes	1.030	Yes	0.004991
10,000	Greedy	398	451	240	Yes	3.411	No (optimal = 3.187)	0.000688
10,000	A*	3,281	3,912	1,742	Yes	3.187	Yes	0.007156
10,000	IDA*	**4,501,500** (did not converge within 3,000 contours)	–	20	**No**	–	—	8.14
100,000	Greedy	1,205	1,340	802	Yes	9.874	No (optimal = 9.295)	0.002103

Thirulogasundar Y - 1923210

n	Algorithm	Nodes expanded	Heuristic evals	Max open/frontier	Found?	Cost	Optimal?	Avg time (s)
100,000	A*	29,870	34,215	17,604	Yes	9.295	Yes	0.067340
100,000	IDA*	**did not converge** (same pathology, worse)	–	–	**No**	–	—	capped (>5 min)
1,000,000	Greedy	3,940	4,228	2,615	Yes	28.41	No (optimal ≈ 27.1)	0.008866
1,000,000	A*	287,415	318,660	164,902	Yes	27.09	Yes	0.741522
1,000,000	IDA*	**did not converge**	–	–	**No**	–	—	capped (>5 min)

(IDA at n=1,000 did converge, in 68 contours, matching A*'s optimal cost exactly; at every larger size it was run with a generous contour cap and still failed to converge within a reasonable wall-clock budget — see the direct quadratic-blow-up measurement below, which is the actual evidence, not merely a timeout.)*

Direct confirmation of the O(N²) pathology (Task 3/7) with real data

At n=10,000, running IDA* with an increased cap of 3,000 contours:

`found=False cost=None nodes_expanded=4,501,500 time=8.14s`

`4,501,500 = 3,000 × 3,001 / 2` **exactly** — i.e. `nodes_expanded_this_contour` was precisely `1, 2, 3, …, 3000` across the 3,000 contours, the textbook signature of quadratic blow-up (Σk for $k=1$ to $K = O(K^2)$) caused by each contour's real-valued f-limit admitting only one additional node before the next limit is needed.

Thirulogasundary - 19232107

Growth pattern across sizes

- *A and Greedy both scale sub-linearly in nodes expanded relative to n** (A*: 371 → 287,415 across 1,000× graph growth — roughly 775×, sub-linear; Greedy: 143 → 3,940, under 30×), because the spatially-informative heuristic keeps both algorithms focused on the geometric neighbourhood of the route rather than the whole graph.
- *A's max_open grows in step with its nodes expanded** (234 → 164,902), confirming the Task 3/5 prediction that A*'s *memory*, not its time, is the resource to watch as the network scales toward and beyond 1,000,000 junctions.
- **Greedy consistently returns a 5–13% more expensive route than optimal** at every tested size (1.142 vs 1.030; 3.411 vs 3.187; 9.874 vs 9.295; 28.41 vs 27.09) while expanding 10–100× fewer nodes than A* — a stable, reproducible speed/quality trade-off, not a one-off anomaly.
- *IDA is the clear outlier**: competitive at n=1,000, then catastrophically non-convergent from n=10,000 onward — precisely because real-valued travel-time costs (an unavoidable feature of a real road network, unlike a grid puzzle with integer step costs) trigger the many-distinct-f-values worst case derived analytically in Tasks 3–7.

TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)

Does the observed growth agree with the theoretical complexity? Yes, strikingly closely for all three algorithms, including in the one place theory predicted a *failure mode* rather than just a growth rate. A*'s node-expansion growth (371 → 287,415, roughly 775× over a 1,000× increase in n) is sub-linear in n, consistent with Task 3's prediction that A*'s cost is governed by heuristic error over the *relevant* geometric neighbourhood, not by total graph size — exactly what a spatially-local, informative heuristic should produce. IDA*'s behaviour is the most theoretically interesting match: Task 7 predicted that continuous, real-valued edge costs push the number of distinct f-values toward the total node count, producing $O(N^2)$ work, and the n=10,000 measurement (`nodes_expanded_this_contour` increasing by exactly 1 each contour, giving the

Thirulogasundar Y - 19232107 3

exact triangular-number formula K(K+1)/2) is not merely *consistent* with that prediction — it is a near-perfect numerical match to it.

Which algorithm scales better as the road network increases? For **node-expansion efficiency and memory**, A* scales best among the three once correctness/optimality is required — its sub-linear growth and bounded, predictable memory profile (even though that memory grows large in absolute terms) make it the most "well-behaved" as n grows. **Greedy scales best in raw speed** (consistently the fewest nodes expanded, e.g. 3,940 vs A*'s 287,415 at n=1,000,000) but this is not free: it pays for that speed with a consistent 5–13% cost penalty at every size, which is also a form of "scaling" that must be reported honestly. *IDA does not scale at all on this workload past the smallest size tested** — the single clearest example in this whole assignment of a textbook algorithm whose standard analysis (favourable on integer-cost puzzles like the 8-puzzle) does not transfer to a different cost structure (real-valued travel times) without modification.

Differences between theory and observation, and their likely causes:

- *A's sub-linear growth is somewhat better than a naive reading of Task 3's bound alone would suggest** — caused by the *spatially-local* graph generator (Task 8's deliberate redesign away from the CO1-style random-attachment graph), which keeps the realised heuristic error small and geographically concentrated; a less spatially-coherent network would show A* scaling closer to its theoretical worst case.
- *IDA's failure to converge is not a discrepancy from theory — it is theory's own predicted worst case, confirmed exactly (K(K+1)/2).** The only "surprise" is how quickly and completely the pathology dominates in practice: Task 7's algebra shows $O(N^2)$ is *possible*; the n=10,000 experiment shows it is not a remote edge case here, but the *typical* behaviour the moment edge costs are continuous rather than integer-valued, which is realistic for travel times measured from GPS/road distance.
- **Greedy's cost penalty is remarkably stable across four orders of magnitude of n** (5.1%, 7.0%, 6.2%, 4.8% suboptimal) rather than growing or shrinking with graph size — suggesting the penalty is a property of the *local* detour-versus-shortcut structure near typical start/goal pairs in this

Thirugnasundar Y - 192321073

generator, not of overall network scale, a useful operational insight: Greedy's quality risk does not obviously worsen just because the city/state network grows.

- No duplicate-state or incorrect-result anomalies were observed; all discrepancies between theory and practice were in the *degree* of a predicted effect, never in its *direction*.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

Recommendation: A*, with Greedy as an explicit, clearly-labelled fallback; IDA* excluded for this domain

Criterion	Greedy	A*	IDA*	Verdict
Route optimality (ties directly to the performance measure: minimum patient wait/transport time)	**Not** guaranteed — measured 4.8–13% worse at every size tested	Always optimal (proven admissible/consistent in Task 1, confirmed in every measured run)	Optimal *when it converges* — but measured to **not converge at all** beyond n=1,000 on this cost structure	A* is the only strategy that reliably delivers the performance measure
Measured nodes expanded at n=1,000,000	3,940 (fastest)	287,415	Did not complete	A* pays a real, bounded cost for its guarantee; IDA* pays an unbounded one
Measured time at n=1,000,000	0.009 s	0.74 s	>5 minutes, still incomplete	A* remains well within an operationally

Thirulogasundar Y – 12221073

Criterion	Greedy	A*	IDA*	Verdict
				acceptable latency for emergency dispatch
Measured memory (max_open) at n=1,000,000	2,615	164,902	O(b·d), small — but irrelevant given it never finishes	A*'s memory is the real resource cost, but it is a known, provisionable quantity (Task 5), unlike IDA*'s unbounded time

Why not IDA*, despite its theoretically attractive O(b\u00b7d) memory? The theoretical advantage assumes a cost structure where contours advance in meaningful jumps; this assumption **does not hold** for real travel-time costs built from continuous road distances and variable speeds, and the experiment shows this is not a rare edge case but the *typical* outcome at every size beyond the smallest tested. Recommending IDA* for a live emergency-dispatch system on the strength of its Big-O memory bound alone, while ignoring this measured, reproducible non-convergence, would be a direct example of letting theory override evidence — exactly what this assignment's evidence rule exists to prevent.

Why A* over Greedy, despite Greedy's large speed advantage? The performance measure explicitly named in Task 1's PEAS specification is **minimum travel time / patient outcome**, not minimum computation time — A*'s sub-second runtime even at n=1,000,000 (0.74 s) is already far faster than any plausible real-world constraint on dispatch decision latency, so there is no operational pressure to accept Greedy's 5–13% worse routes merely to save a fraction of a second of *server-side* computation. **Where Greedy remains defensible:** as an explicit, clearly-labelled **fallback** for a situation where A*'s memory footprint genuinely cannot be provisioned (for instance, an extremely resource-constrained in-vehicle fallback router operating without a connection to the main server), accepting a bounded, measured quality cost in exchange for a guaranteed, much smaller memory and time footprint — but this should be a deliberate, documented trade-off decision, not the default.

Thirulogasundar Y - 192321073

Practicality as the network grows further: A*'s measured, sub-linear node-expansion growth (Task 9–10) gives confidence it remains computationally practical well beyond 1,000,000 junctions; the resource to monitor and provision for is memory (164,902 open-list entries already at 1,000,000 junctions), not time — a concrete, actionable operational conclusion directly traceable to the Task 3/5 theory and the Task 9 measurements together.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

Changed requirement: after a city-wide flooding/storm event, an unknown subset of roads become impassable, and the ambulance only discovers a closure when it physically arrives at the junction where that road begins.

A. Impact Analysis (2 Marks)

The original A*/Greedy/IDA* formulation assumed the road graph was **fully known and static** at planning time — the whole route could be computed once, before departure, and then simply driven. That assumption breaks completely: the environment is now **dynamic and only partially observable through direct arrival** at each junction, so a route computed in advance can become invalid mid-journey, and the search problem changes from "find a path once" to "find a path, discover you were wrong, and find a new one, possibly repeatedly, while the clock (and the patient's condition) keeps running."

B. Evaluation of the Original Design (2 Marks)

None of Greedy, A* or IDA* as implemented in Tasks 1–9 can handle this directly — each computes one fixed route from a fixed start and simply returns it, with no mechanism to detect a closure or to resume search from wherever the ambulance actually is. **Two natural adaptations exist**, both evaluated with real code below: (1) **re-planning A***, which re-runs the full informed search from the ambulance's current position every time a closure is discovered, retaining the global map knowledge gained from every previous attempt; (2) an **online, LRTA*-style agent**, which never computes a whole route in advance, instead greedily picking

Thirulogasundar Y - 192321073

the best-looking next junction at each step using a locally-updated heuristic table, discovering closures only as it tries to act on them.

C. Adaptation or Redesign (2 Marks)

Decision: Adapt (modify), not redesign — both variants reuse A* itself as a subroutine (re-planning A*) or reuse the same admissible straight-line heuristic as a starting table (LRTA*), so the core informed-search machinery from Tasks 1–9 survives; what changes is the *control loop* wrapped around it.

*Re-planning A (implemented):**

```
def replanning_astar_agent(adj, coords, start, goal, closed_set,
max_steps=100000):
    known_closed = set(); current = start; total_driven_cost = 0.0
    total_nodes_expanded = 0; replans = 0; path = [start]; steps = 0
    while current != goal and steps < max_steps:
        steps += 1
        filtered = build_filtered_adj(adj, known_closed)      # remove roads known-
closed so far
        res = astar(filtered, coords, current, goal)          # re-plan from CURRENT
position
        total_nodes_expanded += res['nodes_expanded']; replans += 1
        if not res['found']:
            return {'found': False, 'total_driven_cost': total_driven_cost,
                    'total_nodes_expanded': total_nodes_expanded, 'replans': replans,
'path': path}
        route = res['path']; discovered_closure = False
        for i in range(len(route) - 1):
            u, v = route[i], route[i + 1]; key = frozenset((u, v))
            if key in closed_set and key not in known_closed:   # discover ONLY on
arrival at u
                known_closed.add(key); current = u; discovered_closure = True; break
            else:
                total_driven_cost += next(c for (nbr, c) in adj[u] if nbr == v)
                current = v; path.append(v)
        if current == goal or not discovered_closure: break
```

```
    return {'found': current == goal, 'total_driven_cost': total_driven_cost,
            'total_nodes_expanded': total_nodes_expanded, 'replans': replans, 'path':
path}
```

*Online LRTA-style agent (implemented):**

```
def lrta_star_agent(adj, coords, start, goal, closed_set, max_steps=200000):
    H = {}; known_closed = set(); current = start; total_driven_cost = 0.0
    total_decisions = 0; steps = 0; path = [start]
    while current != goal and steps < max_steps:
        steps += 1; best = None
        for (nbr, c) in adj[current]:
            key = frozenset((current, nbr))
            if key in known_closed: continue
            total_decisions += 1
            val = c + H.get(nbr, straight_line_heuristic(coords, nbr, goal))
            if best is None or val < best[0]: best = (val, nbr, c, key)
        if best is None:
            return {'found': False, 'total_driven_cost': total_driven_cost,
                    'total_decisions': total_decisions, 'steps': steps, 'path': path}
        val, nbr, c, key = best
        if key in closed_set:
            known_closed.add(key); H[current] = val; continue    # discovered closed:
re-decide, don't move
        H[current] = val; total_driven_cost += c; current = nbr; path.append(nbr)   #
LRTA* update + move
    return {'found': current == goal, 'total_driven_cost': total_driven_cost,
            'total_decisions': total_decisions, 'steps': steps, 'path': path}
```

Which testing principles/search properties gain importance: completeness under a *changing* graph (not just a static one) and the cost of *replanning itself* (nodes expanded per re-plan, not just per original plan) both become first-class measured quantities, alongside the original route-cost/optimality measure.

D. Evidence-Based Defence (2 Marks)

Experiment (n=10,000 road network, hospital=junction 0, accident site=junction 9,999, closures sampled randomly at 0%, 5%, 10% of all 11,497 roads, closure_seed=1):

Closure fraction	Roads closed	Baseline optimal cost (no closures)	Re-planning A*: driven cost	Re-planning A*: total nodes expanded	Re-planning A*: # re-plans	Online LRTA*: driven cost	Online LRTA*: total decisions	Online LRTA*: steps taken
0%	0	3.187	**3.187** (optimal)	3,281	1	64.25	1,845	459
5%	574	3.187	8.565	9,230	3	202.02	5,035	1,299
10%	1,149	3.187	**3.187** (optimal — this run's random closures happened not to intersect the optimal route)	3,281	1	882.04	19,083	5,529

Thirulogasundari Y - 19

What the experiment shows: *re-planning A consistently finds a far cheaper actual driven route than the online agent** — by roughly 20× at 0% closures (3.19 vs 64.25) and growing to over 275× at 10% closures (3.19 vs 882.04), because re-planning A* retains and reuses *global* map knowledge (including every closure discovered so far) every time it replans, while the online agent only ever reasons

one step ahead with a slowly-updating local heuristic table, so it can take long, winding detours before its H-values converge toward anything useful. The 10% row's identical cost to the 0% row for re-planning A* is not an error but a reminder that *re-planning A's driven cost depends on whether that run's specific closures happen to lie on the optimal route*, not simply on the closure percentage — the 5% run's closures did intersect the optimal route (forcing 3 re-plans and a real detour), while this particular 10% run's did not.

A genuine limitation, also measured (bonus finding, different random closure seed): with `closure_seed=99` at 10% closures, **both agents correctly report found: False** — investigation (a full-knowledge BFS ignoring discovery order) confirmed the hospital and accident site become **genuinely disconnected** once all 1,149 sampled closures are applied, not a search bug. Re-planning A* detects this efficiently and conclusively (A* itself proves no path exists from the agent's last position after only 5 re-plans), while the online agent, having no global view, cannot prove unreachability — it simply exhausts its step budget (643,208 decisions, 200,000 steps) without ever concluding failure cleanly. *This is itself a concrete, evidence-based argument for re-planning A over a purely online agent in a safety-critical dispatch system:* when a route genuinely does not exist, the control room needs to know that quickly and certainly (to dispatch a different ambulance or alert the crew), not have a vehicle wander indefinitely.

Final decision: adopt **re-planning A*** as the production design for the closure scenario, with the plain (non-adaptive) A* from Tasks 1–11 as its subroutine, unchanged. The measured 20–275× driven-cost advantage over the online alternative, together with its clean, provable failure detection when a route genuinely does not exist, outweighs its one real cost — repeated A* invocations (up to several thousand extra node expansions per discovered closure, Task 9's per-call cost now paid multiple times) — which remains comfortably affordable given A*'s already-demonstrated sub-second runtime even on a 1,000,000-junction network (Task 9–11).

Thirulogasundar Y - 192321073